



Course code: 22AIE311

Semester: 06

Course Title: Software Engineering

Batch: AIE - A

Group Number: 16

Group Members:

Name	Roll number
D. Surya	AM.SC.U4AIE23032
K. Bruhadesh Varma	AM.SC.U4AIE23039
R. Teja	AM.SC.U4AIE23053

ABSTRACT

Quick-commerce platforms require instant delivery to remain competitive. Current delivery systems using human couriers face challenges including traffic congestion, high operational costs, and limited scalability. Drones offer a potential solution for faster delivery operations. Companies like Amazon Prime Air and Zipline have demonstrated drone delivery feasibility, while research efforts focus on flight control and hardware optimization.

However, existing solutions face significant limitations. Commercial implementations remain restricted to specific areas, while academic research addresses individual components such as path planning or battery management in isolation. There is a notable gap in comprehensive software platforms that integrate automated drone assignment, battery constrained route optimization, regulatory compliance with airspace restrictions, weather-dependent planning, and failure recovery mechanisms. Current tools either focus on simulation without real-world constraints or provide proprietary solutions inaccessible for research purposes. This project proposes a web-based simulation system for autonomous drone delivery operations. The system will assign drones to delivery orders using optimization algorithms that evaluate battery capacity, payload weight, distance, weather conditions, and restricted airspace zones. PostgreSQL with spatial extensions will be used for geographic data storage and route validation. Machine learning models will predict battery consumption based on operational parameters. The platform will simulate drone fleet operations to evaluate assignment strategies under varying conditions including high order volumes and system failures. This project aims to develop an open-source software framework for autonomous drone logistics in quick-commerce applications.

Overview of Architecture

The system uses a client-server architecture with three main components:

- **Frontend (Client Side):** A web application that allows users to create orders, view drone assignments, and monitor simulated operations through interactive maps.
- **Backend (Server Side):** A server that processes optimization requests, executes machine learning models for battery prediction, validates routes against airspace regulations, and manages the simulation environment.

- **Database and Storage:** PostgreSQL database with spatial extensions for storing order data, drone information, and geographic zones.

System Flow & Components

(A) User Interaction via Web

Users access the platform through a web browser where they can perform three main tasks:

1. **Order Creation:** Select pickup and delivery locations on a map, specify package weight, and submit delivery requests.
2. **Drone Assignment Monitoring:** View assigned drones, calculated routes on the map, and compliance status with airspace regulations.
3. **Real-Time Tracking:** Monitor simulated drone movements, battery levels, and delivery progress during simulation execution.

(B) Data Communication Between Client and Server

- **HTTP REST API:** Handles order creation, drone status queries, and assignment requests. The server responds with JSON data containing assignment results and status information.
- **WebSocket Connections:** Enables real-time telemetry updates from server to client, allowing live tracking of drone positions and status changes.
- **Spatial Queries:** The system queries the PostGIS database to validate if planned flight paths intersect with restricted airspace zones.

(C) Backend Processing and Decision Making

- **Route Optimization:** OR-Tools library solves the assignment problem by finding optimal drone-to-order assignments that minimize delivery time while satisfying battery range, payload capacity, and airspace restriction constraints.
- **Battery Prediction:** A machine learning model estimates battery consumption for each route based on distance, package weight, and weather conditions.
- **Failure Handling:** When a simulated drone failure occurs, the system triggers reassignment by selecting an available backup drone and recalculating the route.

(D) Real-Time Results Delivery to User

- **Immediate Feedback:** Assignment results show which drone is assigned, the planned route, estimated delivery time, and predicted battery consumption.
- **Live Updates:** Simulated drone positions, battery percentage, and flight status are streamed to the browser in real-time.
- **Analytics Dashboard:** Users can view summary statistics including delivery time, battery usage, and system performance metrics.

Technology Stack Selection

- **Frontend (Client Side)**
 - Web Framework: React.js for building the user interface
 - Mapping Library: Leaflet.js for displaying interactive maps with OpenStreetMap data
 - Real-Time Communication: Socket.IO for receiving live telemetry updates
 - UI Components: Material-UI for responsive interface design
- **Backend (Server Side & AI Processing)**
 - API Framework: FastAPI (Python) for building REST API endpoints
 - Optimization Library: Google OR-Tools for solving Vehicle Routing Problems
 - Machine Learning: scikit-learn for battery prediction models
 - Real-Time Messaging: WebSockets for streaming telemetry data
- **Database and Storage**
 - Primary Database: PostgreSQL with PostGIS extension for storing orders, drones, and spatial data
 - Caching Layer: Redis for storing weather data and simulation states
 - Deployment: Docker containers for consistent

deployment

- **External Services and Models**

- Optimization Engine: OR-Tools VRP solver with constraints for battery range, payload limits, and airspace restrictions
- Battery Model: Random Forest regression model using distance, weight, and weather parameters
- Weather Integration: OpenWeatherMap API for real-time weather conditions
- Compliance Engine: PostGIS spatial functions for route validation against restricted airspace zones